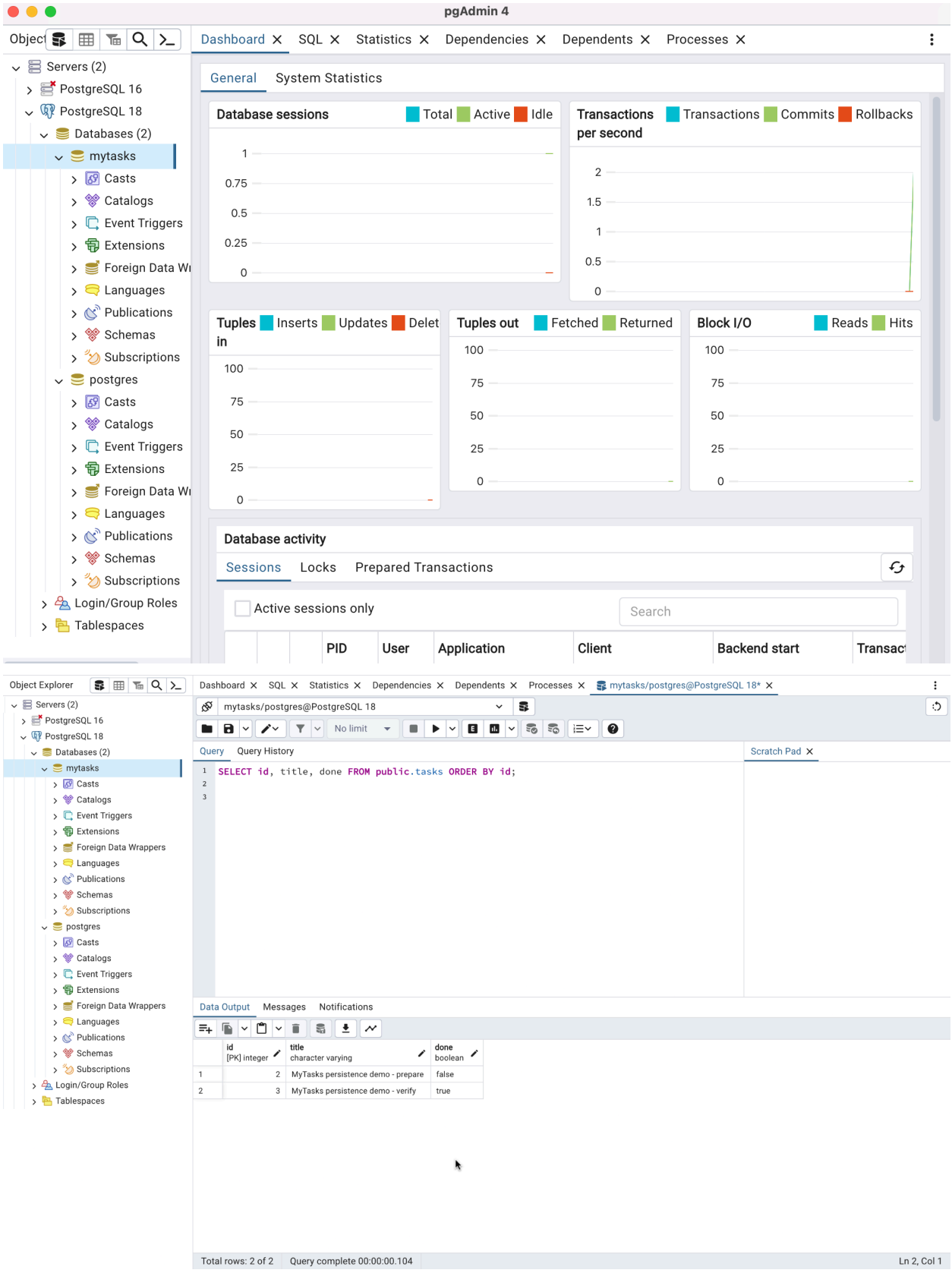
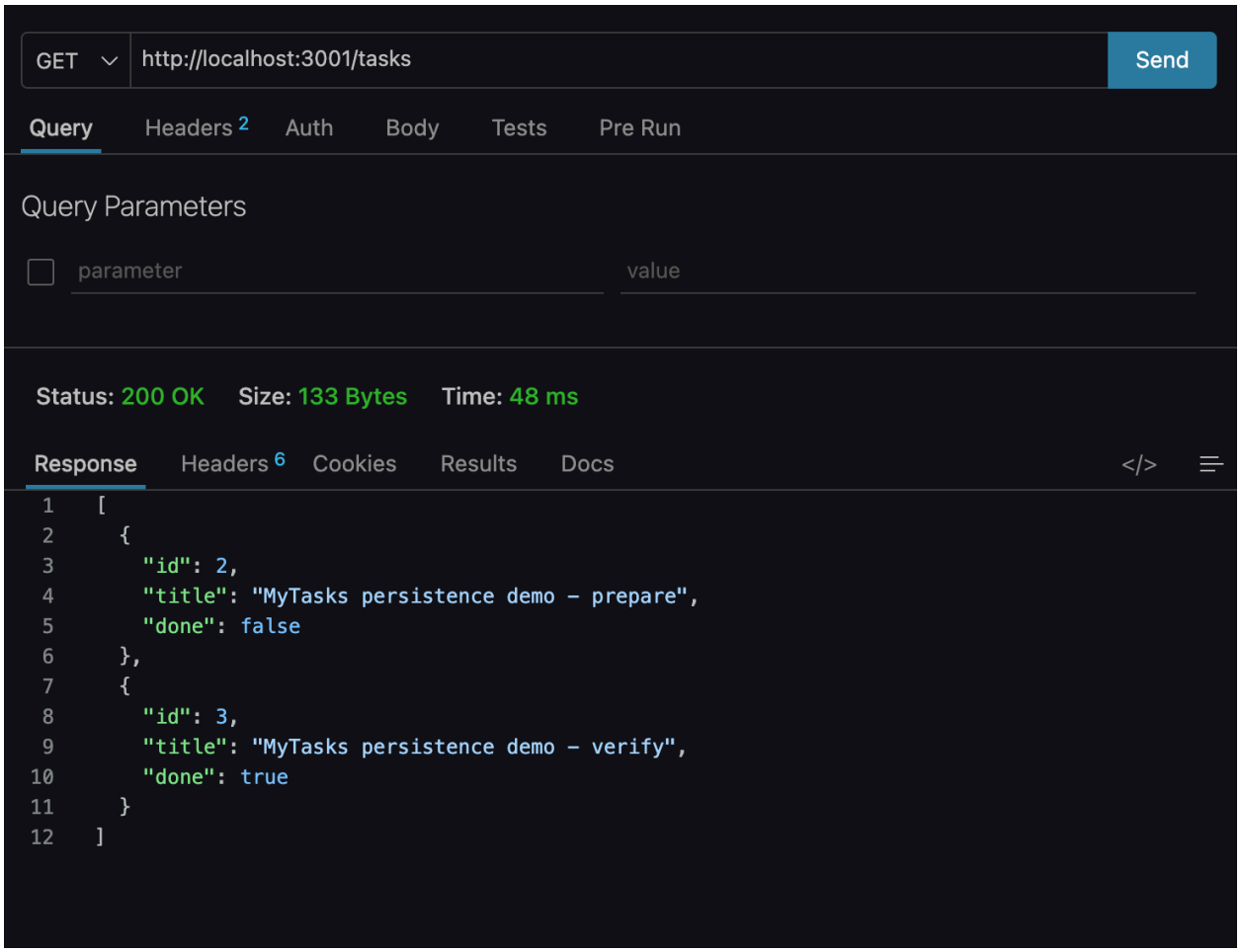
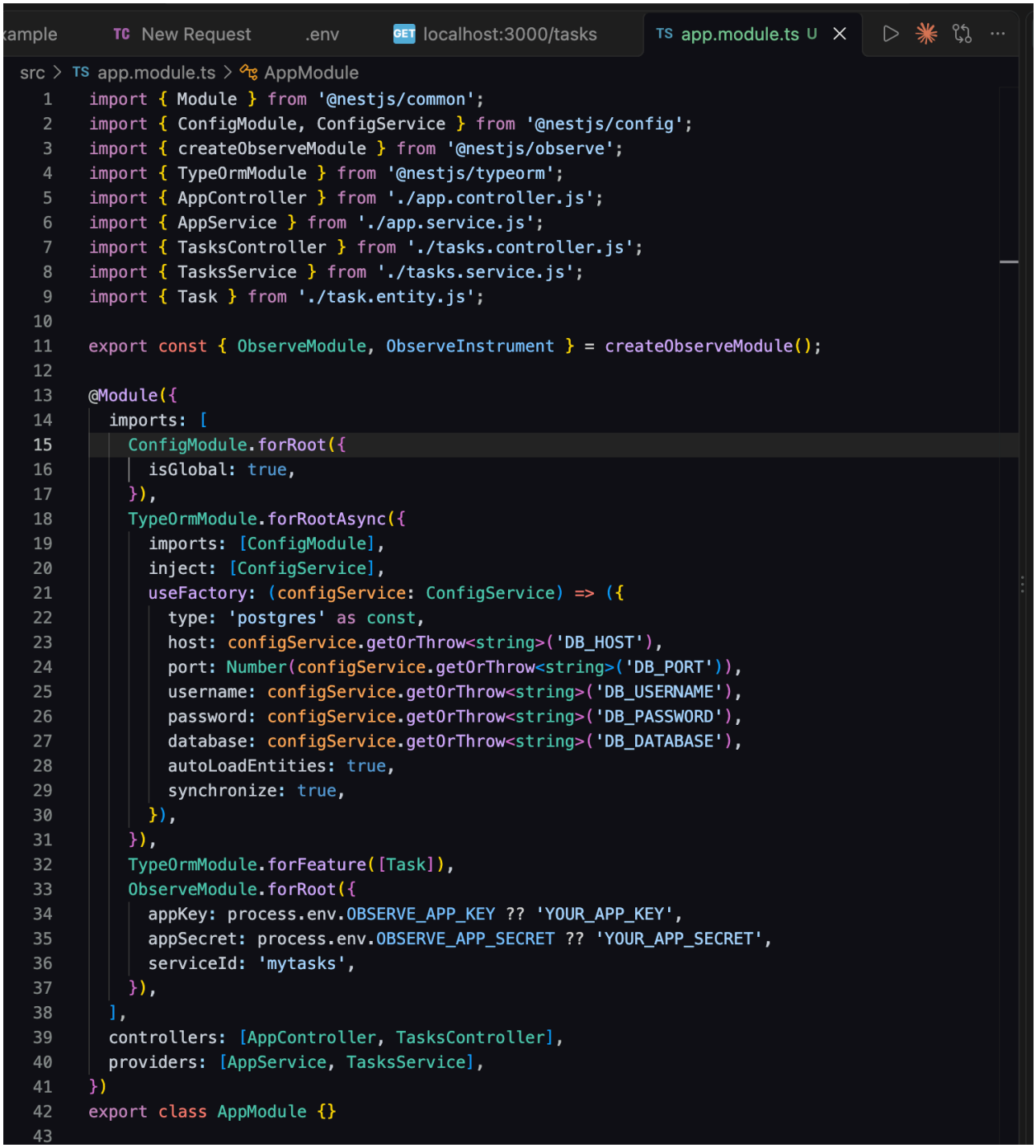
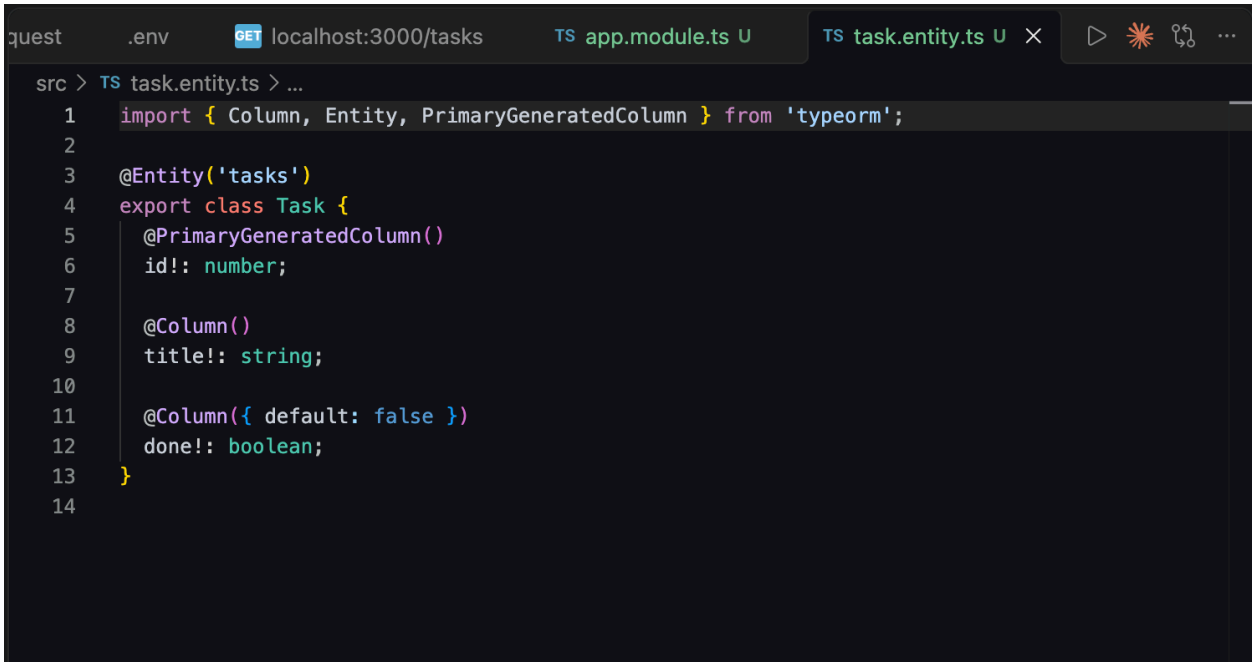

pgAdmin 4
Object Explorer
Servers (2)
PostgreSQL 16
PostgreSQL 18
Databases (2)
mytasks
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrappers
Languages
Publications
Schemas
Subscriptions
postgres
Login/Group Roles
Tablespaces
Dashboard
SQL
Statistics
Dependencies
Dependents
Processes
mytasks/postgres@PostgreSQL 18*
General
System Statistics
Database sessions
Total
Active
Idle
Transactions per second
Transactions
Commits
Rollbacks
Tuples in
Inserts
Updates
Delet
Tuples out
Fetched
Returned
Block I/O
Reads
Hits
Database activity
Sessions
Locks
Prepared Transactions
Active sessions only
Search
PID
User
Application
Client
Backend start
Transac
mytasks/postgres@PostgreSQL 18
No limit
Query
Query History
Scratch Pad
SELECT id, title, done FROM public.tasks ORDER BY id;
Data Output
Messages
Notifications
id
[PK] integer
title
character varying
done
boolean
2
MyTasks persistence demo - prepare
false
3
MyTasks persistence demo - verify
true
Total rows: 2 of 2
Query complete 00:00:00.104
Ln 2, Col 1


GET
http://localhost:3001/tasks
Send
Query
Headers 2
Auth
Body
Tests
Pre Run
Query Parameters
parameter
value
Status: 200 OK
Size: 133 Bytes
Time: 48 ms
Response
Headers 6
Cookies
Results
Docs
</>
[
{
"id": 2,
"title": "MyTasks persistence demo - prepare",
"done": false
},
{
"id": 3,
"title": "MyTasks persistence demo - verify",
"done": true
}
]


ample
TC New Request
.env
GET localhost:3000/tasks
TS app.module.ts U
src > TS app.module.ts > AppModule
import { Module } from '@nestjs/common';
import { ConfigModule, ConfigService } from '@nestjs/config';
import { createObserveModule } from '@nestjs/observe';
import { TypeOrmModule } from '@nestjs/typeorm';
import { AppController } from './app.controller.js';
import { AppService } from './app.service.js';
import { TasksController } from './tasks.controller.js';
import { TasksService } from './tasks.service.js';
import { Task } from './task.entity.js';
export const { ObserveModule, ObserveInstrument } = createObserveModule();
@Module({
imports: [
ConfigModule.forRoot({
isGlobal: true,
}),
TypeOrmModule.forRootAsync({
imports: [ConfigModule],
inject: [ConfigService],
useFactory: (configService: ConfigService) => ({
type: 'postgres' as const,
host: configService.getOrThrow<string>('DB_HOST'),
port: Number(configService.getOrThrow<string>('DB_PORT')),
username: configService.getOrThrow<string>('DB_USERNAME'),
password: configService.getOrThrow<string>('DB_PASSWORD'),
database: configService.getOrThrow<string>('DB_DATABASE'),
autoLoadEntities: true,
synchronize: true,
}),
}),
TypeOrmModule.forFeature([Task]),
ObserveModule.forRoot({
appKey: process.env.OBSERVE_APP_KEY ?? 'YOUR_APP_KEY',
appSecret: process.env.OBSERVE_APP_SECRET ?? 'YOUR_APP_SECRET',
serviceId: 'mytasks',
}),
],
controllers: [AppController, TasksController],
providers: [AppService, TasksService],
})
export class AppModule {}


quest
.env
GET localhost:3000/tasks
TS app.module.ts U
TS task.entity.ts U
src > TS task.entity.ts > ...
1 import { Column, Entity, PrimaryGeneratedColumn } from 'typeorm';
2
3 @Entity('tasks')
4 export class Task {
5 @PrimaryGeneratedColumn()
6 id!: number;
7
8 @Column()
9 title!: string;
10
11 @Column({ default: false })
12 done!: boolean;
13 }
14